TIERLM · RELEASE 0.6.0 · NOVEMBER 2026

Inside the engine

What's new in tierlm 0.6.0, and what its C++ actually does, step by step, from opening a model file to streaming the reply. With an honest comparison against llama.cpp.

Fast

Every part of a model in the memory where it runs best, settings chosen by measuring on your PC.

Accurate

The same answers as the original model, every piece checked against an independent reference.

Easy

Unzip, pick a model, chat. One click finds the fastest settings or sets up your coding app.

tierlm · an EHTi project · github.com/EHTi-Chad/tierlm · MIT licence
Code written by Claude (Anthropic), directed, tested and debugged on real hardware by Chad Jameson (EHTi). Measurements in this guide were taken on the hardware named next to each one. llama.cpp details were checked against its source code as of 1 November 2026.

Contents

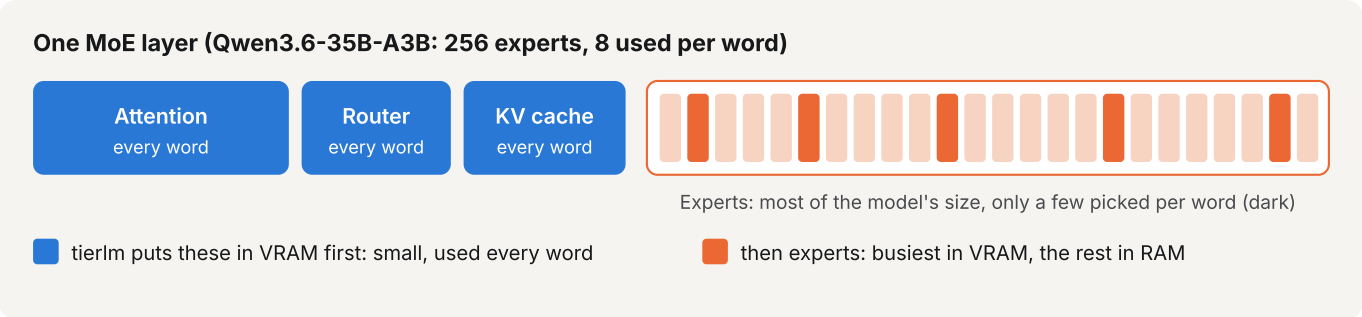
1 tierlm in one page
2 What's new in 0.6.0
3 The speed fix: guessing that checks itself
4 The code map
5 The life of one request
6 Loading: deciding where every weight lives
7 One graph builder for every model family
8 Hot experts: learning what a model uses
9 Keeping the GPU busy
10 Speculative decoding
11 Words in, words out
12 Accuracy: how every piece is checked
13 Ease of use
14 tierlm and llama.cpp, side by side
15 What tierlm changed in ggml
16 What's next

1

tierlm in one page

tierlm runs large language models on your own PC. Its idea is simple: not every part of a model is equally busy, so put the busiest parts in the fastest memory.

A model is a stack of layers. In each layer, **attention** (how words relate to each other), the **router** and the **context memory** (KV cache) are used for every single word. In a **mixture-of-experts** (MoE) model, each layer also holds dozens or hundreds of small "expert" networks, and the router picks only a few of them for each word. Most of a MoE model's size is experts that sit idle most of the time.



Speed comes from where the busy parts live. VRAM is read at about 1,800 GB/s on an RTX 5090; DDR5 system RAM at about 90 GB/s.

33 tok/s
17 GB Qwen3.6-35B-A3B on an 8 GB laptop RTX 3070 (13 tok/s when split by whole layers)

94 tok/s
37 GB Qwen3.6-35B-A3B Q8_0 on an RTX 5090 + 64 GB RAM, in a normal chat

270 tok/s
gpt-oss-20b on an RTX 5090 in a chat with 0.6.0 (135 before this release's fix)

tierlm is its own engine. It borrows **ggml**'s math kernels (the fast matrix routines for CPU, Vulkan and CUDA, with two small, documented changes), and writes everything that decides *what* to compute and *where*: the model file reader, the placement planner, the tokenizers, the model graphs, the KV cache, speculative decoding, the server and the web app. It does not use llama.cpp.

2

What's new in 0.6.0

A new model family, a way for the model to work on your project files, faster replies, and one-click setup for coding apps.

Models

gpt-oss

OpenAI's open models (gpt-oss-20b, gpt-oss-120b) and models built on them, such as HyperNova-60B. They needed six new pieces: attention sinks, sliding-window layers, biases throughout, a clamped expert activation, exact YaRN position stretching, and OpenAI's own "harmony" chat format and o200k tokenizer. Checked against a reference written from Hugging Face's gpt-oss code.

Easy

The workspace

Pick a project folder in the chat settings. The model can list, read and search its files, and suggest changes. Each change shows as a diff with **Apply change** and **Skip**; nothing is written until you click Apply, and only if the file hasn't changed since. No commands run, nothing outside the folder is touched.

Fast

Prompt lookup, with a self-check

When a reply repeats text from the conversation (rewriting a file, quoting code), tierlm guesses the repeat and the model checks it in one pass. New in this release: it times its own guesses and pauses whenever they're slower than not guessing, so it can never slow a chat down. Section 3.

Fast

Choosing each word, 8× faster

After the model scores every word in its vocabulary, one word is picked. gpt-oss has 201,088 words; tierlm used to sort through all of them every time. It now keeps only the likely candidates in one pass: the same picks, 1.7 ms → 0.2 ms per word.

Easy

Set up an app in one click

API → Set up an app: choose opencode, Open WebUI, Continue, Cline or a Python script, and where it runs. tierlm writes the settings for the loaded model (name, context length, reply limit, tool calling, the right address). For opencode, a PowerShell command saves the file for you.

Accurate

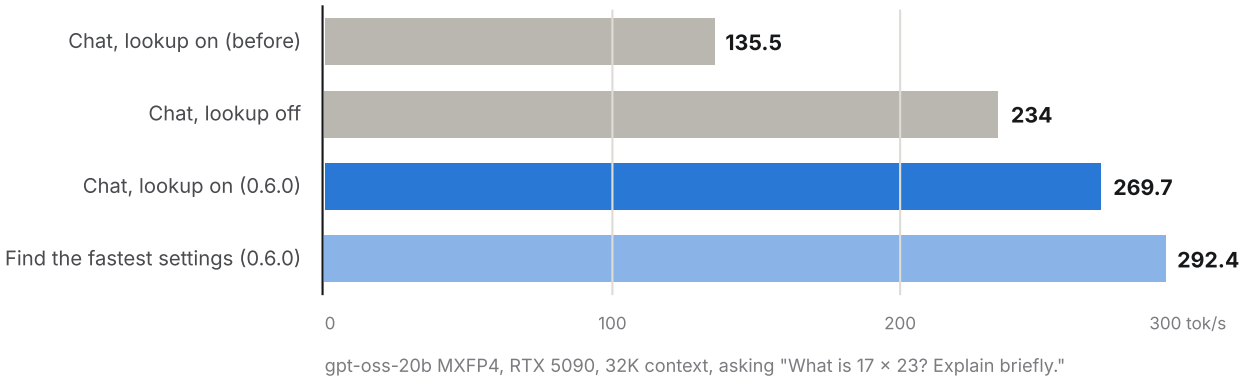
Find the fastest settings tells the truth

The tuner now also asks a short worked question, the kind of reply where guessing used to hurt. On the RTX 5090 it reports 292 tok/s for gpt-oss-20b, and a chat measures 250–270, instead of about 250 promised and 135 delivered.

Also new: maths in replies drawn as real formulas (bundled, works offline; code and dollar amounts are left alone) · tool definitions keep the order your app sent them, as the models' own templates expect · a full-size console view · a calmer speed figure.

The night the tuner and the chat disagreed

Testing gpt-oss-20b on an RTX 5090: **Find the fastest settings** promised about 250 tok/s, a chat delivered 135. Turning prompt lookup off brought the chat to 234, which pinned the cause. Two changes later, the chat and the tuner agree.



gpt-oss-20b MXFP4, RTX 5090, 32K context, asking "What is 17 × 23? Explain briefly."

The remaining gap between the tuner (three questions, longer replies) and one short chat reply is normal variation: short replies spend proportionally more time starting up.

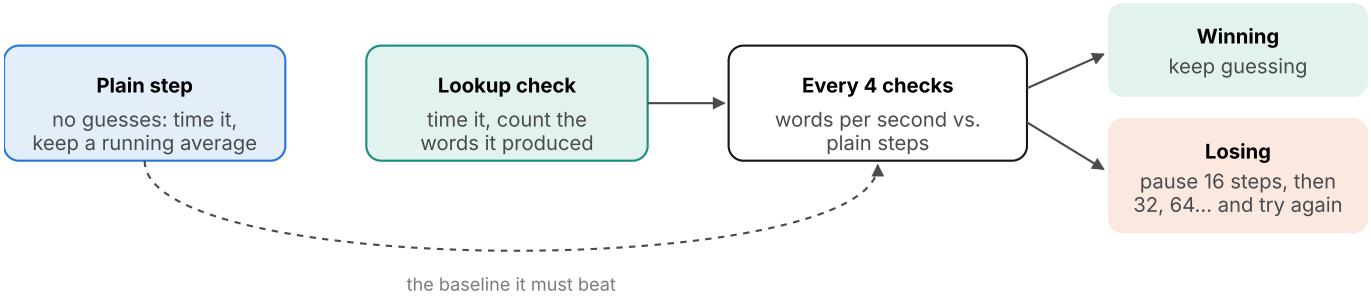
3

The speed fix: guessing that checks itself

Speculative decoding is one of the biggest speed tools there is. It is also easy to get wrong in a way that quietly costs speed.

A model normally makes one word per pass over its weights. **Prompt lookup** looks for the last few words of the reply earlier in the conversation; if they appeared before, the words that followed them are a good guess for what comes next. The model then checks the word it just chose *and* all the guessed words in one pass. Every guess it agrees with is a free word; at the first disagreement, the model's own choice is kept and the rest is undone. The reply is the same either way.

The catch: a check is not free. On a MoE model each guessed word picks its own experts, so checking 8 guesses can read far more weights than one word. In reasoning text, short matches are everywhere (the question's numbers, the model's own markers) but the text then goes a different way. On the 5090, 21 of 38 guesses in one reply were wrong, and each wrong one cost more than it saved.



The timing decides only *whether* to guess, never what the reply says. If the reply starts repeating text later (a code rewrite), lookup comes back and keeps its gain.

```
// src/speculate.cpp, end of each step: is lookup paying for itself?
const double dt = seconds_since(t0);
if (k == 0) t_plain_ = t_plain_ > 0 ? 0.9 * t_plain_ + 0.1 * dt : dt;   // plain step: running average
else if (from_lookup > 0) {
    look_s_ += dt; look_tok_ += m + 1; look_n_++;           // m guesses kept + the model's own word
    if (look_n_ >= 4 && t_plain_ > 0) {                             // a round of 4 checks
        if (look_tok_ / look_s_ < 1.0 / t_plain_) {               // slower than plain steps:
            pause_ = pause_len_; pause_len_ = min(pause_len_ * 2, 1024);   // pause, longer each time
        }
        else pause_len_ = 16;
        look_s_ = 0; look_tok_ = look_n_ = 0;
    }
}
```

Measured, same output with and without lookup	Lookup off	Lookup on
gpt-oss-20b, RTX 5090, short worked answer (before the self-check)	234	135.5
gpt-oss-20b, RTX 5090, short worked answer (0.6.0)	234	**250–270**
Qwen3-0.6B, 2 CPU threads, renaming a word in 1.6 KB of code	12.5	**18.6**
Qwen3-0.6B, 2 CPU threads, short maths answer (lookup holds back)	15.5	15.0

Picking the word: one pass instead of a sort

After each pass the model gives a score to every word in its vocabulary. Sampling settings (top-k 40, top-p, min-p, temperature) only ever choose among the top few dozen. The old code computed a probability for all 201,088 gpt-oss words and partially sorted them; the new code walks the scores once, keeping the best 40 in a small heap, and only then computes probabilities. The repetition penalty is applied to the few recent words afterwards, which is exact: a word outside the kept set is beaten by at least 40 words the penalty didn't touch.

```
for (size_t i = 0; i < nv; ++i) {                    // h: a heap with its worst candidate first
    const float v = lg[i];
    if (h.size() < want) { h.push_back({v, i}); push_heap(h.begin(), h.end(), better); }
    else if (v > h.front().first) { pop_heap(...); h.back() = {v, i}; push_heap(...); }
}
```

tools/server.cpp, `sample()`. Checked against the old code on 9,000 random cases: identical picks (only exact ties, where the old order was arbitrary, can differ). 1.73 → 0.20 ms per word with top-k 40, on the test machine's CPU.

4

The code map

About 7,300 lines of C++ and one 2,200-line web page. Each part has one job.

The app

web/index.html · chat, dashboard, setup

tools/server.cpp · API, tuner, chat loop

agent.h · harmony.h · workspace.h

The engine (src/)

model_map
reads the GGUF header: every tensor's role

planner
what fits where (VRAM, RAM, SSD)

tokenizer
text ⇄ token ids, own splitters + Unicode tables

backends
loads the best CPU engine, CUDA, Vulkan

model (1,560 lines): the heart
tier loader · one graph builder for 11 families · KV cache · hot-expert split · MTP draft layer · batched prompts · undo for speculation

speculate
guess (MTP, lookup) → check in one pass → keep → self-timing

Borrowed (third_party/, MIT)

ggml · tensor maths for CPU, CUDA, Vulkan, quantized formats (2 small changes) plus KaTeX in the web page, for drawing maths

cpp-httplib · HTTP

nlohmann/json · JSON

Arrows of dependency run downwards: the app uses the engine, the engine uses ggml. Nothing in src/ knows about HTTP or the web page.

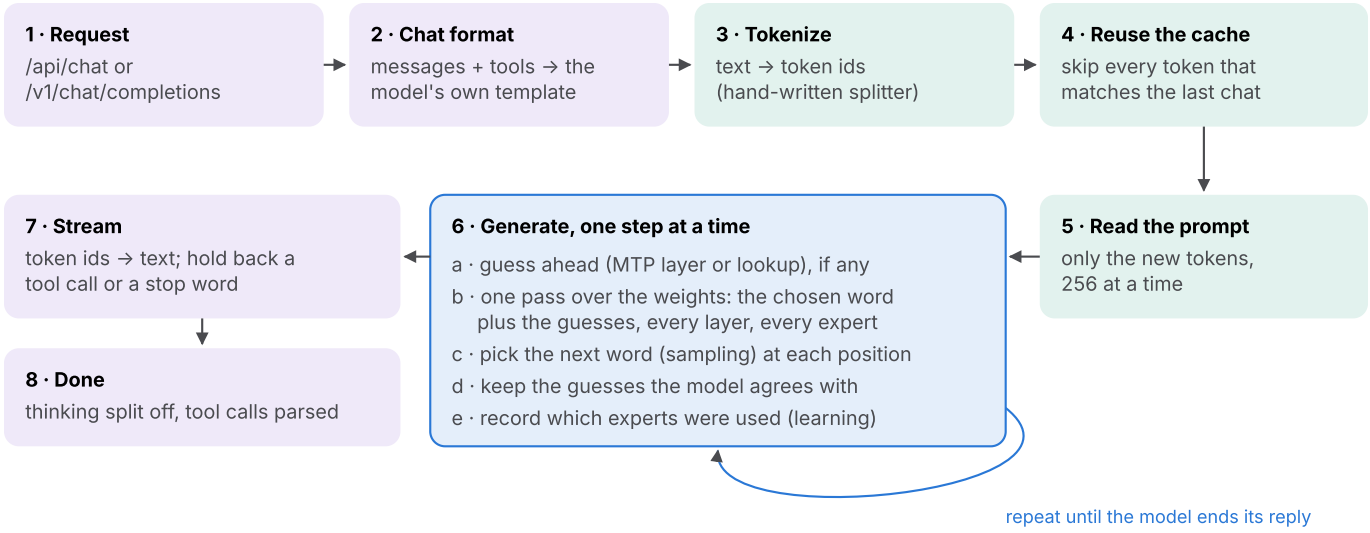
File	Lines	What it does
`src/model.cpp`	1,564	Loads a model, decides where each weight lives, builds the compute graph for every family, runs it, keeps the KV cache
`tools/server.cpp`	2,101	Web server and API, the chat loop, sampling, Find the fastest settings, benchmark, workspace endpoints
`src/tokenizer.cpp` + `unicode_tables.h`	906	BPE and SentencePiece tokenizers with hand-written splitters (GPT-2, Llama 3, Qwen, o200k)
`tools/workspace.h`	371	The workspace: list, read, search, diffs and safe apply
`tools/harmony.h`, `tools/agent.h`	613	Chat formats and tool calls: OpenAI harmony, Qwen XML and Hermes JSON
`src/speculate.cpp`	180	Speculative decoding: guess, check, keep, undo, self-timing
`src/backends.cpp`	191	Picks the best CPU engine for this processor, loads GPU engines whose drivers exist
`src/model_map.cpp`, `planner.cpp`, `gguf_split.cpp`	208	Reads GGUF headers (including split files) and plans placement

For scale: llama.cpp's `src`, `common` and `tools` folders hold about 237,000 lines (counted 1 November 2026). That is not a criticism of either: llama.cpp supports far more models, platforms and features. tierlm's small size is a choice: one person can read all of it, and every line serves the three goals.

5

The life of one request

What happens between you pressing Send and the first word appearing, in order.



Purple: tools/server.cpp and the chat-format headers. Green and blue: the engine in src/.
One request runs at a time; a second one waits a few seconds, then gets a "busy" answer.

Step 4 matters more than it looks

A chat sends the whole conversation every time. tierlm compares the new token list with what is already in the KV cache and only reads what changed. In a long chat that turns a 20,000-token prompt into a 50-token one. It is also why tool definitions must keep their order: if an app's tool list came back reordered, the prompt would differ near the top and the whole cache would be read again. tierlm stores tools as ordered JSON for exactly that reason.

Step 6, in code

```
// tools/server.cpp, generate(): the speculative loop (simplified)
int32_t x = pick(prompt_logits);                    // the first word
for (;;) {
    if (is_end(x)) break;                             // harmony: <|return|> or <|call|>
    std::vector<int32_t> prod = spec->step(x, threads, pick, is_end, &experts_used);
    eng.add_usage(experts_used, K);                 // hot experts learn from every chat
    add(x);  for (j...) add(prod[j]);                // decode and stream each kept word
    x = prod.back();                                // the model's own next word
}
```

The same `generate()` serves the web chat, the OpenAI-compatible API and, importantly, **Find the fastest settings** and the **Benchmark**. The tuner doesn't run a simplified loop: it runs this one, with your sampling settings and a fixed seed, so its numbers are chat numbers.

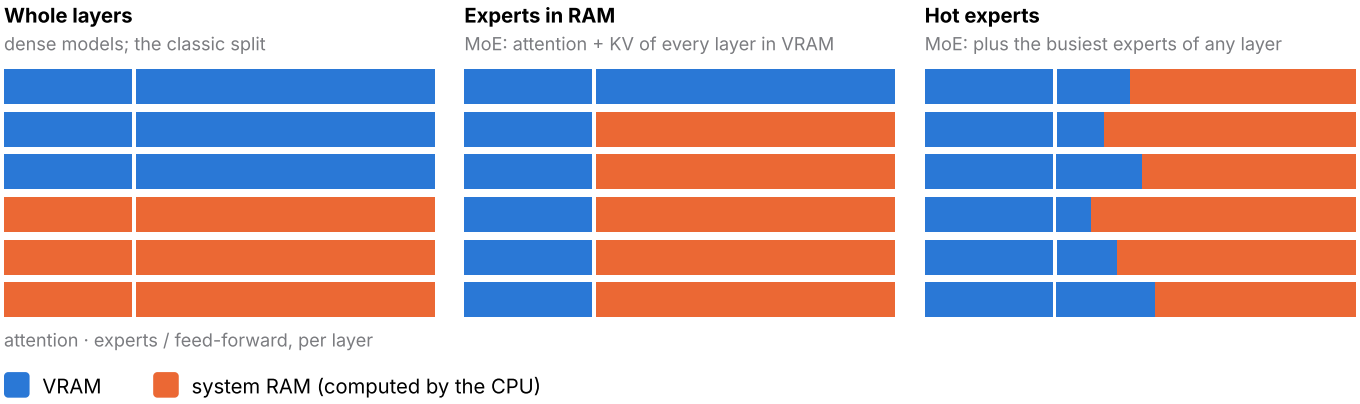
6

Loading: deciding where every weight lives

`Model::load` runs in five stages: pick the device, read the headers, decide placement, stream the weights, build the cache and scheduler.

1. **Pick the device.** The biggest dedicated GPU, else an integrated one, else the CPU. With several GPUs of the same kind, each becomes another VRAM tier.
2. **Read the headers only.** Shapes and types of every tensor, no data yet, so placement can be planned before a single byte is read. Split files (`-00001-of-00003.gguf`) are read as one.
3. **The tier loader decides placement** (below).
4. **Stream the weights** tensor by tensor from the file straight into the buffer where the planner put them. Weights the CPU will multiply are stored in ggml's *repacked* layout where one exists, rows interleaved for wide CPU instructions: prompts read 3.5× faster on the CPU, results unchanged.
5. **KV cache and scheduler.** The cache goes in VRAM when it fits. ggml's scheduler runs each operation on its device and falls back to the CPU for anything a GPU can't do. Compute memory is reserved once for the largest graph, so it isn't re-planned every word.

Three ways to split a model that doesn't fit



Qwen3.6-35B-A3B (17 GB) on an 8 GB laptop RTX 3070: 13.0 tok/s by whole layers, 33.3 with experts in RAM, 29.7 with hot experts. Which one wins depends on the PC, so tierlm measures all three.

With experts in RAM, the GPU does attention for every layer, and for each word the CPU reads only the 8 experts the router picked in each layer. That is the key to big MoE models on small GPUs: the CPU's share of each word is small, and it is the part RAM is fast enough for.

```
// src/model.h: what the tier loader decided, shown on the setup screen and dashboard
struct Placement {
    uint64_t gpu_weights, cpu_weights;          // bytes of weights in VRAM and in system RAM
    int attn_gpu, ffn_gpu;                      // layers whose attention / experts are in VRAM
    std::vector<std::vector<int>> hot;         // per layer: experts kept in VRAM by the hot-expert placement
    std::vector<Gpu> gpus;                      // per GPU, with several cards
    int repacked;                               // CPU weights stored in the faster repacked layout
};
```

7

One graph builder for every model family

Llama, Mistral, Mixtral, Qwen2/3, Qwen3.5/3.6/3.8, Qwen3-Next, OLMoE and gpt-oss are all variations on one design. tierlm describes the differences as switches and builds them all with one function.

```
// src/model.h (excerpt): the switches that make one builder run eleven families
struct Hparams {
    bool rope_neox;                // rotate halves (NeoX) instead of adjacent pairs (original Llama)
    enum QkNorm { kNone, kFull, kPerHead } qk_norm;
    bool qkv_bias;                 // biases on Q/K/V projections (Qwen2, gpt-oss)
    bool expert_renorm;            // chosen experts' weights sum to 1
    bool expert_softmax_first;     // softmax then pick; false: pick, then softmax (gpt-oss)
    bool swiglu_oai;               // gpt-oss: clamped SwiGLU
    int  n_swa;                    // sliding window: these layers see only the last n_swa tokens
    std::vector<float> rope_ff;     // YaRN, computed exactly as Hugging Face does
    bool hybrid;                   // Qwen3.5/3.6: Gated DeltaNet layers
    int  n_nextn;                  // built-in MTP draft blocks
};
```

A mixture-of-experts layer, in tierlm's code

This is the core of every MoE model tierlm runs. Each line adds one node to a ggml graph; ggml runs the graph on whatever device holds each weight.

```
// src/model.cpp, Model::ffn_block (shortened)
ggml_tensor* scores = ggml_mul_mat(c, L.router, h);             // score every expert, for every token
if (L.router_b) scores = ggml_add(c, scores, L.router_b);        // gpt-oss: router bias
ggml_tensor* probs = hp.expert_softmax_first ? ggml_soft_max(c, scores) : scores;
ggml_tensor* order = ggml_argsort(c, probs, GGML_SORT_ORDER_DESC); // best first
ggml_tensor* sel   = /* the top n_expert_used rows of order */;
if (want_routing) g.orders.push_back(ggml_cont(c, sel));        // for the dashboard and for learning
// ... weights of the chosen experts (renormalised, or softmax over the picks for gpt-oss) ...
up   = ggml_mul_mat_id(c, up_exps,   h3, ids);                   // each token through ITS experts only
gate = ggml_mul_mat_id(c, gate_exps, h3, ids);
if (biases) up = ggml_add_id(c, up, L.up_xb, ids);               // gpt-oss: each expert's own bias
act  = hp.swiglu_oai ? ggml_swiglu_oai(c, gate, up, 1.702f, 7.0f)
                     : ggml_mul(c, ggml_silu(c, gate), up);
down = ggml_mul(c, ggml_mul_mat_id(c, down_exps, act, ids), weights);   // weight each expert's output, then sum
```

The chosen expert ids are read back from the GPU once per word for all layers together. Until 0.5.1 that happened once per layer, which was part of why chats ran slower than the benchmark.

Family	Feed-forward	Special pieces
Llama 2/3, Mistral, TinyLlama	dense	Llama 3.1 RoPE factors, SentencePiece or Llama 3 BPE
Mixtral, Qwen3 MoE, OLMoE	experts	QK-norm (per head or full width), renormalised or raw expert weights
Qwen3.5 / 3.6 / 3.8 (dense, MoE), Qwen3-Next	dense or experts + shared expert	3 of 4 layers are Gated DeltaNet (a running memory instead of attention); built-in MTP draft layer
gpt-oss, HyperNova-60B	experts (MXFP4)	attention sinks, sliding window on even layers, biases everywhere, clamped SwiGLU, exact YaRN, harmony format

8

Hot experts: learning what a model actually uses

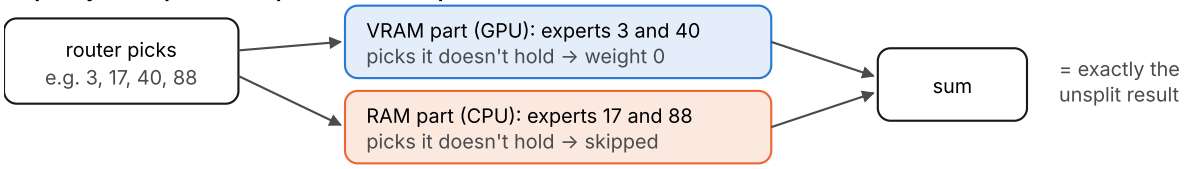
Routers have favourites. On real models a handful of experts per layer take most of the picks. tierlm learns which, per model, and keeps those in VRAM.

1 · Learn
every prompt and reply adds the router's picks to a per-model count, saved in tierlm-usage/ (counts only)

2 · Place
at load, rank all experts by count; fill VRAM with the busiest of any layer, never-used ones last

3 · Measure
Find the fastest settings compares it with plain experts-in-RAM and turns it on only where it wins

A split layer computes each pick where its expert lives



Each part gets one extra all-zero expert. On the CPU, a pick held by the other part gets id −1 and is skipped (one of tierlm's two changes to ggml).

Why it's off by default. On an RTX 3070 laptop with Qwen3.6-35B-A3B, the VRAM experts did 71% of the expert work, yet the plain split was still faster (33.3 against 29.7 tok/s): splitting a layer adds copies and bookkeeping. On a different PC the balance can go the other way. So tierlm doesn't guess: **Find the fastest settings** learns from a short mixed text when nothing has been learned yet (writing, code, maths, translation), measures both, and keeps the winner.

> **How this differs from llama.cpp (checked 1 Nov 2026):** llama.cpp recently added an optional GPU cache for experts kept in RAM (`--moe-cache-mib` , off by default). It works while the model runs: recently and repeatedly used experts are copied into a VRAM cache on demand (a segmented LRU). tierlm's approach is the opposite trade: usage is learned across all your chats, saved, and placed once at load, so nothing is copied while you chat. The two could be combined: a learned starting set plus a small on-demand cache. That's on the list in section 16.

9

Keeping the GPU busy

A fast GPU can be starved by small things: thousands of tiny kernel launches, memory re-planned every word, the CPU waiting on the wrong instruction set.

CUDA graphs, and why tierlm's graphs hold still

Producing one word launches thousands of small GPU jobs. ggml can record them once and replay them with a single launch (a CUDA graph), but only while the graph's shapes and memory addresses stay exactly the same, and ggml's build option for it is off unless you turn it on. tierlm turns it on and builds its graphs to hold still:

- The KV cache is written **by index** (`ggml_set_rows` with the positions as an input), not by a view that moves every word.
- Attention reads the cache in **steps of 256 positions**, with the positions not written yet masked out, so the shape only changes every 256 words.
- One-word steps, several-word steps (speculation) and the MTP draft layer are built in **separate memory**, so each keeps its own recording.

On a laptop RTX 3070 the GPU had been spending about 22 ms per word mostly launching jobs. With graphs replayed: Qwen3.6-35B-A3B with experts in RAM went from **21.7 to 33.3 tok/s**, same results.

The right CPU engine for every processor

Release builds carry ggml's CPU engine compiled for each x86 generation: plain x86-64, SSE4.2, AVX, AVX2, AVX-VNNI, AVX-512, Zen 4 and AMX. At start-up `src/backends.cpp` lets ggml score each against the processor and loads the best one. A GPU engine whose driver isn't installed simply doesn't load. One download runs on a 2010 PC and uses AVX-512 on a Ryzen 9000.

Memory planned once

Compute memory is reserved once, for the largest graph tierlm will ever build (a full 256-token batch at the end of the context), instead of being re-planned on every word. Long prompts are read in batches of 256 tokens. Where a GPU can't run an operation, ggml's scheduler sends it to the CPU automatically.

Several GPUs

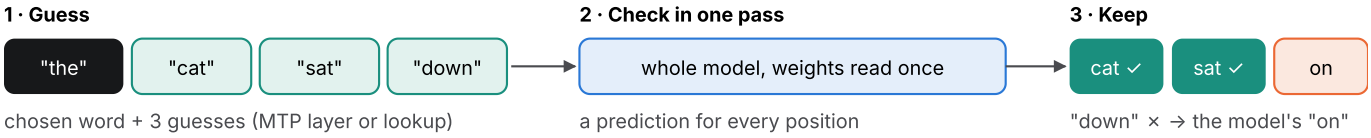
Every GPU of the same kind as the best one becomes another VRAM tier (one card visible to both CUDA and Vulkan isn't counted twice). Whole layers fill the first GPU, then the next, then RAM; with experts in RAM, each layer's attention goes to a GPU first, then its experts if there's room. The tests run every model split across two stand-in GPUs and RAM at many sizes.

> **A finding worth sharing:** ggml moves big batches of RAM weights to the GPU by itself when reading long prompts (from 32 tokens). That's good for prompt speed, but it means the GPU needs valid expert ids even for experts it doesn't hold, and CUDA's expert multiply assumes a token never picks the same expert twice. tierlm's split layers give such picks distinct, unused experts with weight 0. Both findings are written up in `docs/GGML_CHANGES.md` for other projects.

10

Speculative decoding

When part of a model is in RAM, speed is set by how fast RAM can be read. Speculation gets several words out of each read.



chosen word + 3 guesses (MTP layer or lookup) — a prediction for every position — "down" × → the model's "on"

4 · Undo what wasn't kept

Attention layers simply forget the extra keys and values (`Model::accept`). Gated DeltaNet layers (Qwen3.5/3.6) can't rewind their running memory, so the check keeps a snapshot per position and restores the right one, along with the convolution's last inputs.

Result: 3 words from one read of the weights. Every word is still the model's own choice at its position, so the reply is the same as without speculation at temperature 0 (the tests check this at every undo depth), and follows the same distribution with sampling.

Two sources of guesses

The MTP draft layer

Qwen3.5 / 3.6 / 3.8 and Qwen3-Next ship a small extra layer trained to predict the next word from the model's last hidden state. It's one layer, usually in VRAM, so guessing is cheap. tierlm loads it from the model file or from Unsloth's separate `mtp-*.gguf`. Find the fastest settings tries 1 to 4 guesses and keeps it on only if it's clearly faster (each guess an MoE model checks brings in its own experts, so MoE models usually win with 1 or 2).

Prompt lookup

Works for every model. When the last 3 to 6 words appeared earlier in the conversation, what followed them there is the guess. The guess length adapts (one word after a miss, twice as long after a full hit) and the self-check from section 3 pauses it when it isn't paying. With an MTP layer, lookup only *extends* MTP's guesses when the two agree.

Measured	Without	With
Qwen3.6-27B (dense, 17 GB), 23 of 64 layers in VRAM, RTX 3070 laptop, MTP 3 guesses	3.0	**5.3**
Qwen3.6-35B-A3B, experts in RAM, RTX 3070 laptop, MTP 1 guess	33.3	**35.3**
Qwen3-0.6B, 8 CPU threads, rewriting 2.6 KB of code with one rename, lookup	10.5	**27.7**

11

Words in, words out

The tokenizer

Before a model sees text, the text is split into pieces (words, numbers, punctuation, spaces) by a pattern each model family defines, and then joined into vocabulary tokens by BPE merges. Most engines run that pattern through a general regular-expression engine or hand-code it; tierlm hand-writes each splitter (GPT-2, Llama 3, Qwen2, Qwen3.5 and, new in 0.6.0, OpenAI's o200k for gpt-oss) using Unicode tables generated from the Unicode database, and tests each against the original pattern: 3,014 of 3,014 strings for o200k, including the tricky case where it splits a word where lower case turns to upper case.

Chat formats, done the model's way

Each model was trained with one exact conversation format. Getting it slightly wrong doesn't crash anything; it just makes the model a bit worse, quietly. tierlm reads the format from the model's own template and builds it exactly:

- **ChatML, Llama 3, Tulu, [INST]** for most families, with tool definitions in the order the app sent them.
- **Harmony** for gpt-oss: a reply is several messages on channels (`analysis` for reasoning, `final` for the answer, `commentary` for tool calls addressed `to=functions.NAME`); tools are described as a TypeScript namespace. tierlm's prompts are compared byte for byte with OpenAI's own template in the tests, tool definitions included.

Tool calls, for coding agents

Coding agents (opencode, Cline, Continue, Cursor's agent mode) send tool definitions and expect structured calls back. Models write calls in their own style: Qwen3.5/3.6 as XML (`<tool_call><function=…>`), others as Hermes JSON, gpt-oss as harmony messages. tierlm parses each, holds the call back from the visible text while streaming, and returns it as an OpenAI `tool_calls` entry, with reasoning in `reasoning_content`. Arguments are kept in the order the model wrote them, so the next request's prompt matches the cache.

The workspace, safely

```
// tools/workspace.h: a change is only written if the file is exactly what the model saw
plan   = workspace::plan(root, args);              // the new content, inside the folder only
diff   = unified_diff(old, plan.new_text);          // shown in the chat with Apply / Skip
hash   = fingerprint(path, old, plan.new_text);     // sent to the page with the diff
// ... you click Apply: the page sends the hash back ...
if (fingerprint(path, read(path), plan.new_text) != hash) return error("the file changed, nothing written");
apply(plan);
```

Every path is resolved and checked to be inside the chosen folder. The workspace endpoints answer only the page on the computer running tierlm: they check that the request comes from this machine, that the Host and Origin headers are tierlm's own, and that a custom header only tierlm's page sends is present. Other devices and other websites get a refusal. Nothing is ever executed.

12

Accuracy: how every piece is checked

tierlm is not built on llama.cpp, so it can't lean on llama.cpp's testing. Every piece is checked against an independent reference written from the original model code, before it ships.

Piece	Reference	Result
Model graphs	Plain NumPy versions of each architecture, run on tiny random models written the way the converters write them (Llama, Llama 3, Mixtral, Qwen2, Qwen3, Qwen3 MoE, OLMoE, gpt-oss)	About 2 in a million difference with a full-precision cache, prompts and one word at a time, with and without flash attention
Hybrid Qwen3.5 / 3.6, Qwen3-Next	NumPy version of Hugging Face's forward pass, every file layout, also split between GPU and RAM	About 1e-5; deliberately wrong conventions fail the test
gpt-oss	Reference written from Hugging Face's `modeling_gpt_oss.py` (sliding window 8, YaRN factor 32)	1.7e-6, including the rounding-sensitive YaRN case below
Speculative decoding	Same model without speculation; scripted, partly wrong guesses at every undo depth; 8-token guesses	Identical output at temperature 0
Splits	Every model split over one or two stand-in GPUs and RAM, by layers, by experts, with made-up hot-expert usage, at many sizes (160 combinations)	Same as unsplit
Tokenizers	Hugging Face tokenizers; the original split patterns on about 3,000 strings each	Exact
Chat formats	OpenAI's gpt-oss template, Qwen3.6's template, rendered by a Jinja engine	Byte-for-byte identical, tools included
Sampling (0.6.0)	The previous sampler, 9,000 random cases	Identical picks

An example: gpt-oss's YaRN, exactly

gpt-oss was trained on 4,096 positions and stretched to 131,072 with YaRN, which blends two rotation speeds over a band of dimensions. ggml's built-in YaRN rounds the edges of that band to whole dimensions (`floorf` and `ceilf` in `ggml_rope_yarn_corr_dims`). gpt-oss's configuration says not to round (`"truncate": false`), and Hugging Face's code follows it. llama.cpp uses ggml's rounded version. tierlm computes the blended frequencies itself, as Hugging Face does, and hands them to ggml as per-dimension frequency factors. The difference is small in everyday use, but tierlm's test model puts the band edges between whole dimensions on purpose: the rounded version fails it, tierlm's passes.

> **Accuracy and speed together.** Every speed feature in tierlm is designed so it can't change the answer: speculation keeps only words the model itself picks; hot experts sum to exactly the unsplit result; repacked CPU weights give the same numbers; the new sampler picks the same words. Where a feature *does* trade accuracy (8-bit or 4-bit KV cache), it is a visible setting, off by default, with its cost stated (8-bit is about 1% from exact).

13

Ease of use

Speed and accuracy only count if people can get to them. tierlm's rule: a person who has never heard of VRAM should get the best result on their PC without reading anything.

Nothing to install

Unzip and double-click `start-tierlm.bat`. The normal graphics driver is enough: CUDA, Vulkan and the right CPU engine come in the zip and tierlm picks between them. A 20 MB "lite" zip for Intel Arc, AMD Radeon or no GPU.

A setup screen, not flags

It shows the hardware it found and the GGUF files in your folder (LM Studio's folder works as is), says which are supported, and places each model automatically. No layer counts, no tensor patterns.

Find the fastest settings

One click loads the model and measures, on your PC, the settings that only change speed: how to split, CPU threads, speculation. It keeps the winner for that model. It measures a real chat, so the number it shows is the number you get.

You can see what it's doing

A live view of speed, context, memory and an expert map showing which experts the router picks and where they live. A console with the engine's log, full size on demand.

Connect your apps in one click

An OpenAI-compatible API with tool calling, and **Set up an app** writes the configuration for opencode, Open WebUI, Continue, Cline or Python. An Allow other devices switch and an optional API key control access.

A workspace with guard rails

The model works on your project folder, and every change waits for your click. Maths shows as maths, thinking folds away, tool steps show as small cards.

Still rough, honestly: Windows shows a warning on first run until tierlm is code-signed; there's no model download inside the app yet; images (for example scanned documents) and running two models at once are not supported yet. All are on the list in section 16.

14

tierlm and llama.cpp, side by side

llama.cpp is the most widely used local engine, and runs inside apps such as LM Studio. Both it and tierlm run on ggml. Every llama.cpp detail below was checked in its source code on 1 November 2026; it moves fast, so some of this will change.

Topic	llama.cpp	tierlm
What it is	A general library and a set of tools (server with a web UI, CLI, benchmarks) for a very wide range of models, including vision and audio, on Windows, Linux and macOS	One app focused on running large models fast on PCs with limited VRAM, with its own web app; 11 model families, Windows download, Linux from source
Kernels	ggml (the same project)	ggml, with two small documented changes
Fitting a model to memory	`--fit`, on by default, estimates memory and decides how many layers go to VRAM; for MoE models, layers past that point keep everything but their experts on the GPU, with the experts in RAM. Manual control with `-ngl`, `--n-cpu-moe`, `-ot`	The same idea by default for MoE (attention and cache of every layer in VRAM, experts in RAM), plus whole-layer and hot-expert modes; the mode is chosen by **measuring speed** on your PC, not by estimating memory
Busy experts in VRAM	Optional cache, off by default: recently used experts copied to VRAM while running (segmented LRU)	Usage learned across chats and saved per model; busiest experts placed at load; used only where measured faster
Speculative decoding	Many methods, all off by default: separate draft model, EAGLE3, MTP, DFlash, DSpark and five n-gram variants	MTP draft layer (turned on where Find the fastest settings measures it faster) and prompt lookup (on by default), with lookup timing itself and pausing when it doesn't pay
gpt-oss positions	YaRN through ggml's rounded band edges	YaRN computed exactly as the original (no rounding)
Tuning	`llama-bench` measures settings you choose	Find the fastest settings measures and picks for you, inside the app, using the chat's own code path
Code size	About 237,000 lines (src, common, tools)	About 7,300 lines of C++

What llama.cpp has that tierlm doesn't (yet)

- Far more model families, including vision and audio models (needed for reading scanned documents).
- macOS and Apple GPUs, and ready-made Linux builds.
- More speculative-decoding methods (EAGLE3, DFlash) and an on-demand expert cache.
- Serving several users at once, and constrained output (grammars, JSON schemas).

Where tierlm aims to be different

- **Decisions by measurement.** The settings that only change speed are measured on your PC with your kind of chat, and the app tells you what it chose.
- **Learning from use.** Expert placement improves with every chat on that model.
- **Safe by default.** Speed features are on only when they can't change the answer and, now, only while they actually pay.
- **Exactness**, down to details like gpt-oss's YaRN, and a reference test for every piece.
- **An app, not a toolkit**: nothing to configure by hand to get the fast path.

15

What tierlm changed in ggml

tierlm uses a fork of ggml (EHTi-Chad/ggml, branch `tierlm`) with two small changes, both documented line by line in `docs/GGML_CHANGES.md` with a patch file other projects can use.

Change 1: the CPU skips expert id −1

In a hot-expert split layer, the RAM part shouldn't compute experts that live in VRAM. Giving those picks id −1 and having the CPU's expert multiply skip them means the CPU computes only what it holds, instead of wasted work on a zero-weight dummy.

Change 2: optional timing per scheduler split

With `TIERLM_PROFILE=1`, the scheduler reports how long each device's part of a word took. That's how tierlm found where each word's time goes, and why hot experts lose on some PCs.

Three findings, written up for others

1. **CUDA graphs are off unless you turn them on**, and they only replay while a graph's shapes and addresses hold still (section 9).
2. **CUDA's expert multiply assumes a token never picks the same expert twice.** Split layers must give spare picks distinct experts.
3. **ggml moves big batches of RAM weights to the GPU by itself** when reading long prompts, so the GPU sees expert ids even for the RAM part.

Measured results so far

Hardware	Model	Setting	tok/s
RTX 5090, Ryzen 9 9950X3D, 64 GB	gpt-oss-20b MXFP4	All in VRAM, chat (0.6.0)	250–270
RTX 5090, Ryzen 9 9950X3D, 64 GB	Qwen3.6-35B-A3B Q8_0 (37 GB)	Experts split VRAM/RAM, chat (0.5.1)	93.7
RTX 5080, Ryzen 7 9800X3D, 32 GB	Qwen3.6-35B-A3B Q6_K (27 GB)	Experts in RAM (0.5.0)	65.2
RTX 3070 laptop, 8 GB	Qwen3.6-35B-A3B Q3_K_XL (17 GB)	Experts in RAM + MTP (0.5.0)	35.3
RTX 3070 laptop, 8 GB	Qwen3-30B-A3B Q3_K_M (14.7 GB)	22 of 48 layers in VRAM	27.1
RTX 3070 laptop, 8 GB	Qwen3.6-27B dense Q4_K_M (17 GB)	23 of 64 layers in VRAM + MTP	5.3
Intel Arc 140V (integrated)	OLMoE-1B-7B Q4_K_M	Vulkan, 64-token context	67.8

Speeds vary by 1–2 tok/s between runs on a laptop; replies at temperature 0.7 vary in length. Each result is from the version named.

16

What's next

Grouped by the three goals. The first group is what lets tierlm replace other apps for real document work.

Easy Next release

- **Two models at once**: a small embedding model (such as nomic-embed-text) beside a chat model, with `/v1/embeddings`, for search and document comparison.
- **Reading images**: image input for models that can see (Qwen3.5 / 3.6), so scanned and handwritten documents can be turned into text.
- **Attach a file to a message** (a + button), plus model search and download inside the app.
- **Code signing**, so Windows stops warning on first run.

Fast

- **Learned placement plus an on-demand cache**: start from the experts tierlm has learned are busiest, and keep a small VRAM cache for the ones a particular chat leans on.
- **Faster prompt reading for coding agents**: keep long agent instructions cached between requests; skip thinking on side requests (titles).
- **CPU and GPU on the same layer at once**: a split layer's VRAM and RAM experts in parallel (needs a change to ggml's scheduler).
- **gpt-oss extras**: hot experts with expert biases (for gpt-oss-120b and HyperNova-60B); a short KV cache for the sliding-window layers.
- **The SSD tier**: rarely used experts read from an NVMe drive, for models bigger than VRAM and RAM together.

Accurate

- Real-weight tests for Qwen3-Coder-Next, Mixtral, Qwen2.5 and HyperNova-60B; new families (Gemma, Nemotron, NVFP4 weights), each with a reference test before release.
- A check that Find the fastest settings' estimate matches real chats for every model, not only the ones tested so far.

> **Keeping up:** a weekly report now checks what llama.cpp, LM Studio, Ollama, vLLM, newer engines and new research have added, says what tierlm should adopt, and proposes ideas no one seems to be doing yet, each searched first to make sure it really is new.

tierlm is open source under the MIT licence: github.com/EHTi-Chad/tierlm. Third-party components keep their own MIT licences: ggml, cpp-httplib, nlohmann/json and KaTeX. Model weights are not included and have their own licences. Copyright © 2026 Chad Jameson (EHTi).